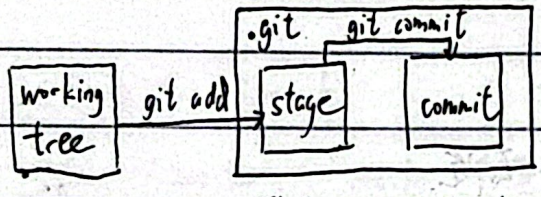
Notes of git 版本管理

<…> 表示文件 "…" 表示文本 …也表示文本

练习文件夹 git_ex 在 ~/Desktop/git_ex

· git 文件管理的三个区 working tree, stage, commit

↳ =wt



过往的版本都在 .git 文件夹中

stage 的作用是打包提交 commit.

工作区　暂存区　提交历史

1° 基础操作

git init 开始用 git 进行版本管理，但要先 cd 到目标文件夹.

git status 看管理状态

git add <…> 把文件放进暂存区.

git commit 提交一个 commit (把现在 stage 中的东西打包提交)

git push 上传到远程仓库

git config --global ↳ user.name "…" 添加提交者姓名/邮箱

↳ user.email "…" 只需做一次即可.

git commit -m "…" 写说明 -m 表示说明，每次都写上心

对于 vim 编辑器，[redacted] :wq 可退出. 一般直接退出.

git log 查看上传日志

↳ git log --oneline 查看上传日志(简化)

clear 清屏

git diff [redacted] 查看 working tree 相对 stage 的改动

git log 查看 commit 历史日志

↓

git diff --cached -- <…> 查看 stage 相对 commit 的改动

2. 回滚 (3种情形)

① 只在 working tree 改动，未进 stage (not staged)

git restore <...> 回滚这个文件

git restore . 回滚这个文件夹内的所有文件

② 改动已进入 stage，未进 commit

需要先把改动从 stage 去除，再从 working tree 去除.

git restore --staged <...> 把这个文件中的改动从 stage 中去除

git restore --staged . 把文件夹中的所有改动从 stage 中去除.

然后再进行 git restore 操作.

③/1 改动已进入 commit (未删除)

HEAD 指向前的 commit，head~1 是前一版 commit，head~2 是...

git reset --hard HEAD~1 把 HEAD 移动到原 head~1 的位置，并把原来的 HEAD 删除，log 也会删除.

--hard 表示在 commit, stage, working tree 一起回滚。(出大问题时用)

--mixed 会保留 working，回滚 stage 和 commit

--soft 会保留 working, stage, 回滚 commit. (有小问题时)

③/2 改动已进入 commit 的另一种方式 (多做一次 commit)

git revert HEAD 用一次新的提交，把文件回滚到未更改的状态，而不删除 HEAD

example: A B C D → E (E 为 HEAD，D 为更新)

在 commit D 后用 git revert HEAD，生成一个 E 作为新 HEAD，E 与 C 内容一致，但号码不同。不会删除 D

好处：可以看当时改了什么.

3° 不想参与版本管理的文件 .gitignore

先建一个 gitignore 文件夹，然后把不想上传的文件放进去.

nano .gitignore → nano 可以创建并打开 .gitignore 文件

作用是：git status 不会显示 .gitignore 中提到的内容。.gitignore 也要提交！

4° 分支创建 branch

HEAD → master master 就是一个分支，也可创建 feature/分支.

习惯：master 分支保存稳定版本。开发新功能用 feature 分支，完成后合并到 master.

git branch ... 添加分支，一般叫 feature/...

git branch 若什么后缀都不加，就列出仓库中的所有分支，包括 master.

*表示当前所在的分支，注意切换 branch。

git switch ... 切换分支

要随时用 git branch 确认当前在哪个分支.

git switch -c feature/aaa 添加并切换分支至 feature/aaa

git switch -c = git branch + git switch = git checkout -b

↑ 功能打包了

5° 分支合并 merge

git merge ... 把别的分支合并到当前分支上。

要记得 merge 之前切换到 master 上。必须把其他分支并入 master.

并完后执行 git log，会显示 HEAD → master, feature/aaa

事合并完成后 feature/aaa 分支会变成合分支，git branch -d ... 删掉

有时要手动解决冲突 conflict

扫描全能王 3亿人都在用的扫描App

↳ both modified

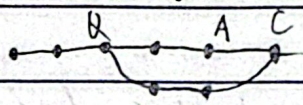
如果两个分支修改了同一行，就会有conflict，因为Git不知道保留哪一行。

有conflict就用 git status 看conflict在哪里，unmerged paths 里的 both modified 就是要手动改的东西，用 nano <--> 改动。

↳找 conflict 标记 >>> === <<<，按要求删掉conflict标记行，改内容

↳保存退出后 git add，git commit

*常用开发方法：



fast forward Q→C

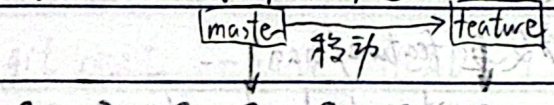
在A git merge 时，C就是新提交的，整合A,B的commit

b° rebase "重置地基"：把自己的分支移到最新的master地基上去

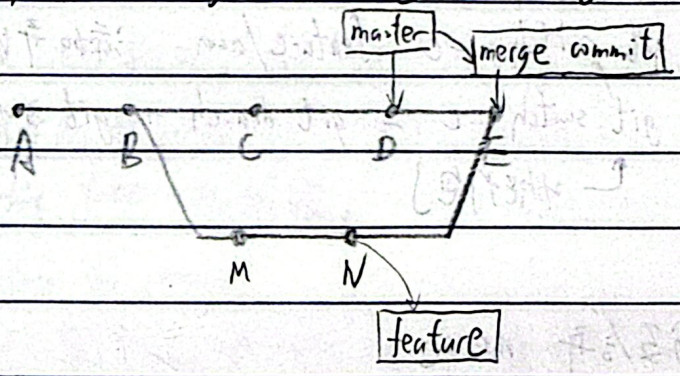
git rebase master

若有conflict，按5°来做。然后 git add <--->，git rebase --continue

若要合回master，再切回master分支，然后 git merge，不需要再commit



把master从D移动到G是fastforward，G天然拥有D,E,F，所以不需要新建一个commit



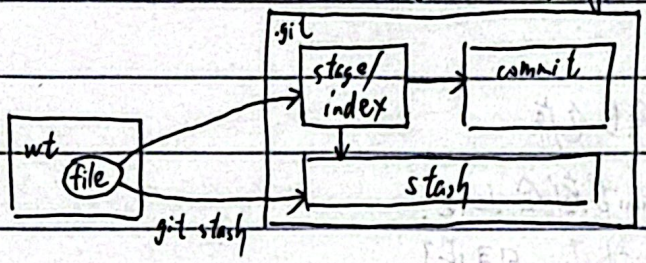
新建一个commit E来结合D和N，

由于D不具有M,N，需要新建commit，需要 git commit。non-fastforward

master和feature/都是由来提交链上的一个节点，而且commit链。

7° stash 把未commit的修改临时收起来，让wt恢复干净/突然切换修bug

git stash 把wt中的改动移到stash中且把stage中的改动移到stash中.



WIP: wt与stage中的改动

hotfix: 紧急修复.

git stash pop 把stash中的东西复制到wt

8° Worktree 创建一个新的工作区.

git worktree add -b feature/aaa ../"..." master

① ②

① 基于master分支创建的 feature/aaa 分支

② 放在"..."文件夹里 .. 表示一次父文件夹.

git config --global core.quotepath false 把收器修显示设为收.

git worktree list 查看worktree

git worktree remove ../"..." 删除"..."worktree.

9° github使用入门

git是本地，github是远程仓库

SSH key 的公钥和私钥，我放在 C:/user/当前电脑/.ssh/ 中

github帐号 pjHe2005 邮箱 204…

ssh-keygen -t ed25519 -C "..." 创建SSH key ①的内容只能抄，不要改.

①

cat ~/.ssh/<...> <...>是公钥的名称 如直接上述去查看公钥

把公钥内容复制到github的 SSH keys 中

ssh -T git@github.com　　若出现 authenticated 表示对

- 在 Github 中如何创建新仓库？ 点右上角头像 → ~~Setting~~ Repos… → 新建，名称为英文

3条教程指导如何上传 git push

git push 把 commit 推送至 Github 线上仓库

↳ 会提交 commit 里的内容，且所有 commit 都会上传。

可以移动 origin，就可以保存到另一个 Github 目录下。

- 如何从 Github 上下载到本地？

git clone "…"　　"…" 中是 Github 的 code → SSH

git log --oneline 可查看 commit 日志。